

Celebal Excellence Internship

Data Engineer

Week 5 Assignment

Name	Yadnyesh Sawant
Student ID	CT_CSI_DE_1015
Domain	Data Engineering
Batch	Batch 1
Start Date	15/6/2026
Status	Active
Contact No	8888047246
Email ID	<u>1272250066@mitwpu.edu.in</u>
College Name	MIT-WPU
Stream	MCA
Passing Out Year	2027

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

Traditional **MapReduce** is a framework used to process large amounts of data, but it has some limitations that make it less efficient for modern applications.

The main limitations are:

- **High Disk Usage:** MapReduce stores intermediate results on disk after every processing stage, which increases execution time.
- **Slower Performance:** Frequent disk read/write operations make MapReduce slower, especially for large datasets.
- **Not Suitable for Iterative Tasks:** Machine learning and data analysis often require multiple iterations, but MapReduce reads data from disk every time, reducing efficiency.
- **Complex Programming Model:** Developers need to write separate Map and Reduce functions, making the code lengthy and difficult to maintain.
- **Limited Real-Time Processing:** MapReduce is mainly designed for batch processing and is not suitable for real-time analytics.

Why Spark is Preferred:

Apache Spark overcomes these limitations by using **in-memory processing**, which stores intermediate data in RAM instead of writing it to disk after every step. This makes Spark much faster and more efficient. Spark also provides easy-to-use APIs in Python, Java, Scala, and R, supports real-time processing, machine learning, SQL, and graph processing within a single framework.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Apache Spark uses In-Memory Computing, which means it stores intermediate data in the system's RAM instead of writing it to disk after every operation. This reduces the time required to read and write data, resulting in much faster processing.

In machine learning, algorithms often need to process the same dataset multiple times before producing the final result. In a disk-based system like MapReduce, the data is read from and written to disk during every iteration, which increases execution time.

In contrast, Spark keeps the data in memory and reuses it for each iteration. This minimizes disk I/O operations and significantly improves performance. As a result, Spark can execute iterative machine learning algorithms much faster than traditional disk-based systems.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Answer: used superset dataset, so columns used in the questions are
"Customer ID", "Order Date"

```
# Total records before removing duplicates
print("Total Records Before:", df.count())

# Count occurrences of each Customer ID and Order Date combination
print("Duplicate combinations before removing duplicates:")
df.groupBy("Customer ID", "Order Date") \
    .count() \
    .filter(col("count") > 1) \
    .orderBy(col("count").desc()) \
    .show(truncate=False)
```

Total Records Before: 9994

Duplicate combinations before removing duplicates:

```
+-----+-----+-----+
|Customer ID|Order Date|count|
+-----+-----+-----+
|SV-20365   |9/20/2017 |14   |
|AC-10615   |9/2/2017  |12   |
|PP-18955   |2/5/2016  |11   |
|WB-21850   |12/11/2016|11   |
|PF-19120   |9/17/2015 |10   |
|NP-18325   |8/9/2015  |10   |
|AG-10270   |9/13/2016 |10   |
|BT-11680   |7/9/2015  |9    |
|GG-14650   |10/31/2014|9    |
|PP-18955   |11/10/2016|9    |
|MP-17965   |4/18/2015 |9    |
|DB-13405   |3/17/2017 |9    |
|GB-14575   |9/21/2015 |9    |
|IM-15070   |12/11/2015|9    |
|SG-20080   |10/31/2015|9    |
|SC-20770   |3/13/2016 |9    |
|KH-16510   |12/8/2017 |9    |
|RP-19390   |9/13/2014 |8    |
|CD-12280   |11/5/2017 |8    |
|NR-18550   |6/3/2017  |8    |
+-----+-----+-----+
```

only showing top 20 rows

```
# Remove duplicates
df_no_duplicates = df.dropDuplicates(["Customer ID", "Order Date"])
```

```
▶ # Total records after removing duplicates
print("Total Records After:", df_no_duplicates.count())

# Check for duplicate combinations again
print("Duplicate combinations after removing duplicates:")
df_no_duplicates.groupBy("Customer ID", "Order Date") \
    .count() \
    .filter(col("count") > 1) \
    .show(truncate=False)
```

```
... Total Records After: 4992
Duplicate combinations after removing duplicates:
+-----+-----+-----+
|Customer ID|Order Date|count|
+-----+-----+-----+
+-----+-----+-----+
```

Q4: Given a DataFrame `df_sales`, write a query to filter for rows where the region is 'West' and then group by `product_category` to find the average `sale_amount`.

Answer:

```
# Filter data for the West region
west_sales = df.filter(F.col("Region") == "West")

# Group by Category and calculate average Sales
avg_sales = west_sales.groupBy("Category") \
    .agg(F.avg(F.expr("try_cast(trim(Sales) AS DOUBLE)")) \
    .alias("Average_Sales"))
```

```
# Display the result
avg_sales.show(truncate=False)
```

```
+-----+-----+
|Category      |Average_Sales  |
+-----+-----+
|Office Supplies|117.48907552370453|
|Furniture      |360.59540420899896|
|Technology     |422.64417449664415|
+-----+-----+
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer:

1. `.na.drop()` removes rows that contain null values in one or more columns. It is useful when incomplete records should not be included in the analysis.
2. `.na.fill()` replaces null values with a specified value instead of removing the entire row. This helps preserve data while handling missing values.

In the Superstore dataset, there is no Status column. Therefore, the Category column is used to demonstrate the `na.fill()` operation.

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100

Answer:

```
from pyspark.sql.functions import count

# Count records for each city
city_count = df.groupBy("City") \
                .agg(count("*").alias("Total_Records"))

# Filter cities having more than 100 records
result = city_count.filter(col("Total_Records") > 100)

# Display the result
result.show(truncate=False)
```

City	Total_Records
Springfield	163
Dallas	157
Philadelphia	537
Los Angeles	747
San Francisco	510
San Diego	170
Detroit	115
Columbus	222
Chicago	314
Seattle	428
New York City	915
Houston	377
Jacksonville	125

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer:

Spark DataFrames are immutable, which means their data cannot be changed after they are created. Instead of modifying the existing DataFrame, every operation such as dropping a column, renaming a column, or filtering data creates a new DataFrame.

During data cleaning, we need to assign the result of each operation to a new DataFrame or overwrite the existing DataFrame variable. The original DataFrame remains unchanged unless we explicitly replace it.

```
# Rename a column
df_renamed = df.withColumnRenamed("Sales", "Total_Sales")

# Drop a column
df_cleaned = df.drop("Postal Code")
```

In the above code:

- withColumnRenamed() creates a new DataFrame with the renamed column.
- drop() creates another new DataFrame without the specified column.
- The original df remains unchanged.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer: Since the Superstore dataset does not contain Age and Subscription columns, the following code filters records where Region = 'West' and Category = 'Technology' to demonstrate the same filtering operation.

```
# Filter records where Region is 'West' and Category is 'Technology'
filtered_df = df.filter(
    (col("Region") == "West") &
    (col("Category") == "Technology")
)
# Display the filtered records
filtered_df.show(10, truncate=False)
```

Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category
BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-10002275	Technology
BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-10002033	Technology
ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	California	94109	West	TEC-PH-10001949	Technology
EH-13945	Eric Hoffmann	Consumer	United States	Los Angeles	California	90049	West	TEC-AC-10003027	Technology
KM-16720	Kunst Miller	Consumer	United States	Los Angeles	California	90004	West	TEC-AC-10004633	Technology
BS-11590	Brendan Sweed	Corporate	United States	Gilbert	Arizona	85234	West	TEC-PH-10002726	Technology
JS-15685	Jim Sink	Corporate	United States	Los Angeles	California	90036	West	TEC-PH-10004093	Technology
LC-16870	Lena Cacioppo	Consumer	United States	Aurora	Colorado	80013	West	TEC-AC-10001552	Technology
TW-21025	Tamara Willingham	Home Office	United States	Scottsdale	Arizona	85254	West	TEC-PH-10001254	Technology
FM-14290	Frank Merwin	Home Office	United States	Los Angeles	California	90032	West	TEC-AC-10002323	Technology

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer:

In Apache Spark, shuffle is the process of moving data between different partitions during operations like `groupBy()`, `join()`, and `reduceByKey()`. When we perform a grouping operation, Spark collects all records with the same key and brings them together into the same partition before performing the calculation.

For example, if we group the Superstore dataset by Category, Spark moves all records belonging to the same category into one partition to calculate values such as total or average sales.

Shuffle is called a wide transformation because the output depends on data from multiple input partitions. Unlike narrow transformations, where data remains in the same partition, a wide transformation requires data to be transferred across different partitions. This data movement increases network communication and processing time, making shuffle one of the most expensive operations in Spark.

Example:

```
category_sales = df.groupBy("Category") \
    .agg(F.round(F.sum(F.expr("try_cast(trim(Sales) AS DOUBLE)")), 2)
    .alias("Total_Sales"))

category_sales.show()
```

Category	Total_Sales
Office Supplies	703502.93
Furniture	733046.86
Technology	835900.07

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Answer

```
# Count records before cleaning
print("Total Records Before Cleaning:", df.count())

# Remove rows where Customer ID is null or Customer Name is empty
df_clean = df.filter(
    col("Customer ID").isNotNull() &
    (trim(col("Customer Name")) != "")
)

# Count records after cleaning
print("Total Records After Cleaning:", df_clean.count())
```

```
•• Total Records Before Cleaning: 9994
    Total Records After Cleaning: 9970
```


Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

```
# Calculate minimum, maximum, and average sales
sales_statistics = df.agg(
    F.min(F.expr("try_cast(trim(Sales) AS DOUBLE)").alias("Minimum_Sales"),
    F.max(F.expr("try_cast(trim(Sales) AS DOUBLE)").alias("Maximum_Sales"),
    F.round(F.avg(F.expr("try_cast(trim(Sales) AS DOUBLE)")), 2).alias("Average_Sales")
)

# Display the result
sales_statistics.show(truncate=False)
```

Minimum_Sales	Maximum_Sales	Average_Sales
0.444	22638.48	234.42

Q14: In the context of cleaning a dataset, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Answer:

The `inferSchema=true` option allows Spark to automatically detect the data type of each column while reading a dataset. This works well when the data is clean, but it can cause problems if the date column contains different date formats.

For example, if a dataset has dates like 01/05/2024, 2024-05-01, and May 1, 2024, Spark may not correctly identify the column as a date. Instead, it may treat the entire column as a string.

If the date column is stored as a string, operations like sorting, filtering, and date calculations may not work correctly. Therefore, it is better to convert the column to the required date or timestamp format manually using functions like `to_date()` or `to_timestamp()`.

```
# Convert Order Date to TimestampType
df_clean = df.withColumn(
    "Order Date",
    to_timestamp(col("Order Date"), "MM/dd/yyyy")
)

# Display the updated schema
df_clean.printSchema()
```

```
root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: timestamp (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: string (nullable = true)
|-- Quantity: string (nullable = true)
|-- Discount: string (nullable = true)
|-- Profit: double (nullable = true)
```

Q15: Write a final processing pipeline that:

1. Filters out duplicates.
2. Fills null prices with 0.
3. Groups by store_id to calculate total revenue.

Answer:

```
[64] # Load the Superstore dataset
df = spark.read.csv(
    "Sample - Superstore.csv",
    header=True,
    inferSchema=True
)

# Step 1: Check total records before removing duplicates
print("Total Records Before Removing Duplicates:", df.count())

# Check how many duplicate records are present
duplicate_count = df.count() - df.dropDuplicates().count()
print("Duplicate Records Found:", duplicate_count)
```

... Total Records Before Removing Duplicates: 9998
Duplicate Records Found: 8

```
[65] # Step 2: Remove duplicate records
df_clean = df.dropDuplicates()

# Check total records after removing duplicates
print("Total Records After Removing Duplicates:", df_clean.count())
```

Total Records After Removing Duplicates: 9990

```
[66] # Step 3: Check null values in Sales before filling
null_count_before = df_clean.filter(col("Sales").isNull()).count()
print("Null Values in Sales Before Filling:", null_count_before)
```

Null Values in Sales Before Filling: 0

```
[67] # Step 4: Fill null values in Sales with 0
df_clean = df_clean.na.fill({"Sales": 0})
```

```
[68] # Step 5: Verify null values after filling
null_count_after = df_clean.filter(col("Sales").isNull()).count()
print("Null Values in Sales After Filling:", null_count_after)
```

Null Values in Sales After Filling: 0

```
[71] # Step 6: Group by Region and calculate total revenue
total_revenue = (
    df_clean
    .groupBy("Region")
    .agg(F.round(F.sum(F.expr("try_cast(trim(Sales) AS DOUBLE")), 2)).alias("Total_Revenue"))
    .orderBy(F.col("Total_Revenue").desc())
)

print("\nTotal Revenue by Region:")
total_revenue.show(truncate=False)
```

Total Revenue by Region:

Region	Total_Revenue
West	710071.19
East	668564.95
Central	497740.53
South	388968.03

